

PERFORMANCE EVALUATION AND DIMENSIONING OF NETWORKS AND SYSTEMS

METI

Lab Report [EN]

Authors:

José Oliveira (103771)

Tiago Videira (116069)

Afonso Mendes (116024)

jose.novais.oliveira@tecnico.ulisboa.pt

tiago.g.videira@tecnico.ulisboa.pt

afonsofmenDES@tecnico.ulisboa.pt

2026/2027 – 1st Semester

Contents

1	Introduction	2
2	A. Random number generators	3
2.1	Exercise 1	3
2.2	Exercise 2	3
3	B. Generation of random variables and stochastic processes	4
3.1	Exercise 3	4
3.2	Exercise 4	6
3.3	Exercise 5	7
4	C. Programming discrete event simulators	9
4.1	Exercise 6	9
4.2	Exercise 7	11
5	D. Estimation of probabilities	13
5.1	Exercise 8	13
6	E. Revisions of statistics	14
6.1	Exercise 9	14
6.2	Exercise 10	17
6.3	Exercise 11	18
7	F. Output data analysis	18
7.1	Exercise 12	18
7.2	Exercise 13	20
7.3	Exercise 14	22
8	Conclusion	24

1 Introduction

Text for the introduction here.

2 A. Random number generators

2.1 Exercise 1

Objective: Generate 1000 random numbers uniformly distributed in $[0, 1]$ using `runif()` and plot their histogram with `hist()`, to check informally whether the generator behaves as expected.

Approach: The following R script generates the sample and plots the histogram.

```
numbers <- runif(10000, min = 0, max = 1)
hist(numbers, main = "Histogram", xlab = "Value", ylab = "Samples",
      col = "lightblue", border = "black")
```

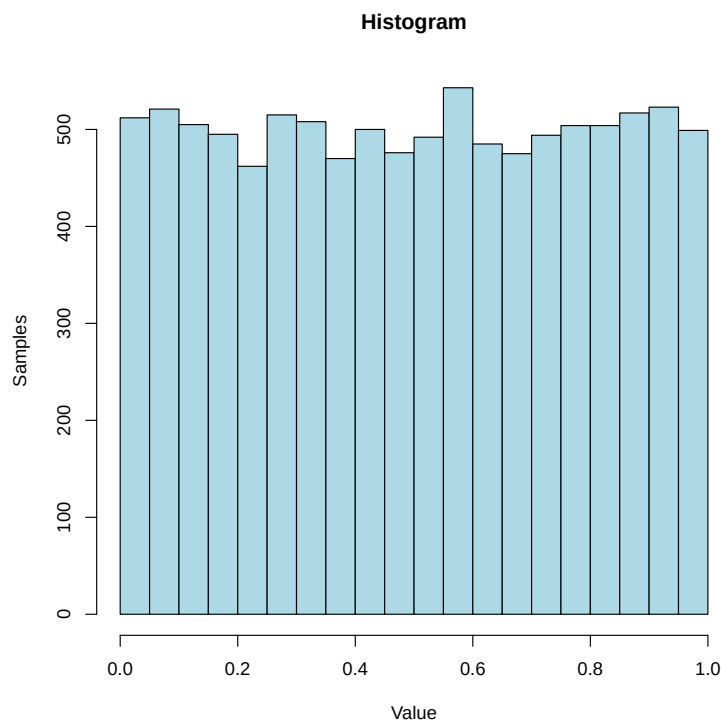


Figure 1: Histogram of 10000 samples generated with `runif()`.

Conclusion: As shown in Figure 1, the samples are approximately evenly distributed across all bins in $[0, 1]$, with no systematic bias toward any sub-interval. This is consistent with the expected behavior of a uniform distribution, confirming that `runif()` is generating numbers as intended.

2.2 Exercise 2

Objective: Verify, using `runif()`, that the sequence generated by a pseudo-random number generator is in fact deterministic, i.e., that it can be reproduced exactly.

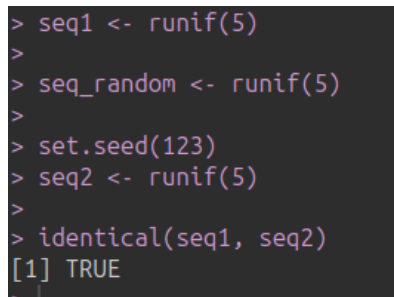
Approach: The `set.seed()` function fixes the internal state of R's random number generator. By setting the same seed before two separate calls to `runif()`, the two resulting sequences can be compared with `identical()` to check whether they match exactly. As a control, a third sequence is generated without resetting the seed, so it is expected to differ.

```
set.seed(123)
seq1 <- runif(5)

seq_random <- runif(5) # generated without resetting the seed

set.seed(123)
seq2 <- runif(5)

identical(seq1, seq2)
```



```
> seq1 <- runif(5)
>
> seq_random <- runif(5)
>
> set.seed(123)
> seq2 <- runif(5)
>
> identical(seq1, seq2)
[1] TRUE
>
```

Figure 2: R console output confirming that `seq1` and `seq2` are identical.

Conclusion: As shown in Figure 2, `identical(seq1, seq2)` returns `TRUE`, confirming that `seq1` and `seq2` are exactly the same sequence, even though they were generated by separate calls to `runif()`. This happens because `set.seed()` resets the internal state of the generator to the same fixed value before each call, so the pseudo-random number generator produces identical output whenever it starts from the same seed. This demonstrates that the sequence is not truly random but deterministic, fully determined by the seed. In contrast, `seq_random`, generated without resetting the seed, follows on from the generator's state after `seq1` and therefore differs from both.

3 B. Generation of random variables and stochastic processes

3.1 Exercise 3

Objective: Develop an R script to generate Bernoulli distributed numbers from a uniform random number generator (`runif()`), and plot a histogram of 1000 samples to informally check that the generator behaves correctly.

Approach: A Bernoulli random variable with success probability p can be generated from a uniform random variable $U \sim \text{Unif}(0, 1)$ by setting the outcome to 1 if $U < p$ and 0 otherwise, using the `ifelse()` function. Here $p = 0.7$ and $n = 1000$ samples were generated.

```
p <- 0.7 # success probability
n_samples <- 1000 # number of samples

u <- runif(n_samples)

bernoulli_numbers <- ifelse(u < p, 1, 0)

hist(bernoulli_numbers,
     breaks = c(-0.5, 0.5, 1.5),
     main = "Bernoulli Numbers Histogram (p = 0.7)",
     xlab = "Outcome",
     ylab = "Frequency",
     col = "lightblue",
     xaxt = "n")

axis(1, at = c(0, 1))
```

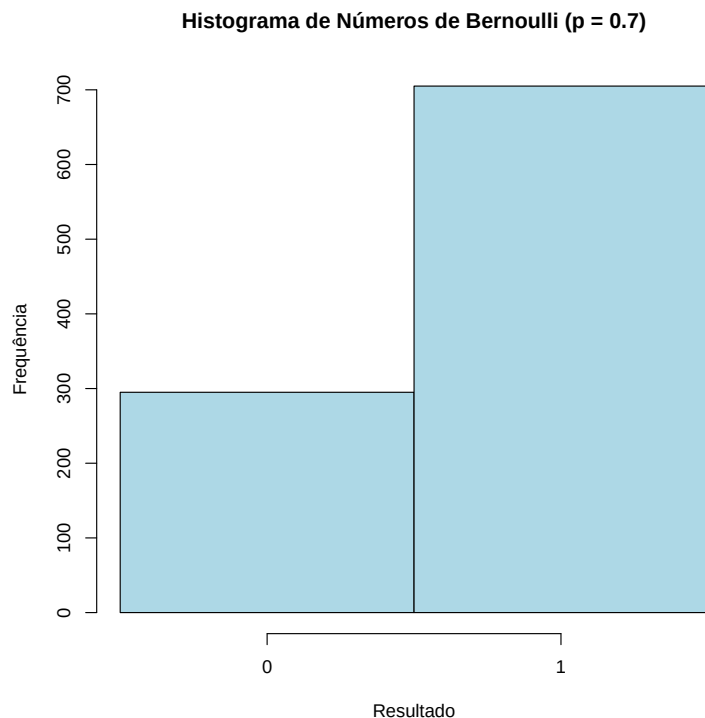


Figure 3: Histogram of 1000 Bernoulli samples with $p = 0.7$.

Conclusion: As shown in Figure 3, out of 1000 samples approximately 700 outcomes are 1

and 300 are 0, matching the expected proportions for $p = 0.7$. This confirms that thresholding a uniform random variable at p is a valid method for generating Bernoulli distributed samples, since $P(U < p) = p$ for $U \sim \text{Unif}(0, 1)$.

3.2 Exercise 4

Objective: Develop an R script to generate exponentially distributed numbers from a uniform random number generator (`runif()`), and plot a histogram of 10000 samples superposed with the theoretical pdf, to informally check that the generator behaves correctly.

Approach: Exponentially distributed samples can be generated from a uniform random variable $U \sim \text{Unif}(0, 1)$ via the inverse transform method: since the exponential CDF is $F(x) = 1 - e^{-\lambda x}$, setting $U = F(X)$ and solving for X gives $X = -\ln(1 - U)/\lambda$. The histogram is plotted with `prob=TRUE` so that its area is normalized to 1, making it directly comparable to the theoretical pdf obtained with `dexp()`, which is superposed using `lines()`. Here $\lambda = 1.5$ and $n = 10000$ samples were generated.

```
lambda <- 1.5 # rate of the exponential distribution
n_samples <- 10000 # number of samples

u <- runif(n_samples)
exp_numbers <- -log(1 - u) / lambda

hist(exp_numbers,
      breaks = 50,
      prob = TRUE,
      main = "Exponential Distribution (lambda = 1.5)",
      xlab = "Value",
      ylab = "Density",
      col = "lightgray",
      border = "white")

x_vals <- seq(0, max(exp_numbers), length.out = 500)
pdf_vals <- dexp(x_vals, rate = lambda)

lines(x_vals, pdf_vals, col = "red", lwd = 2)
```

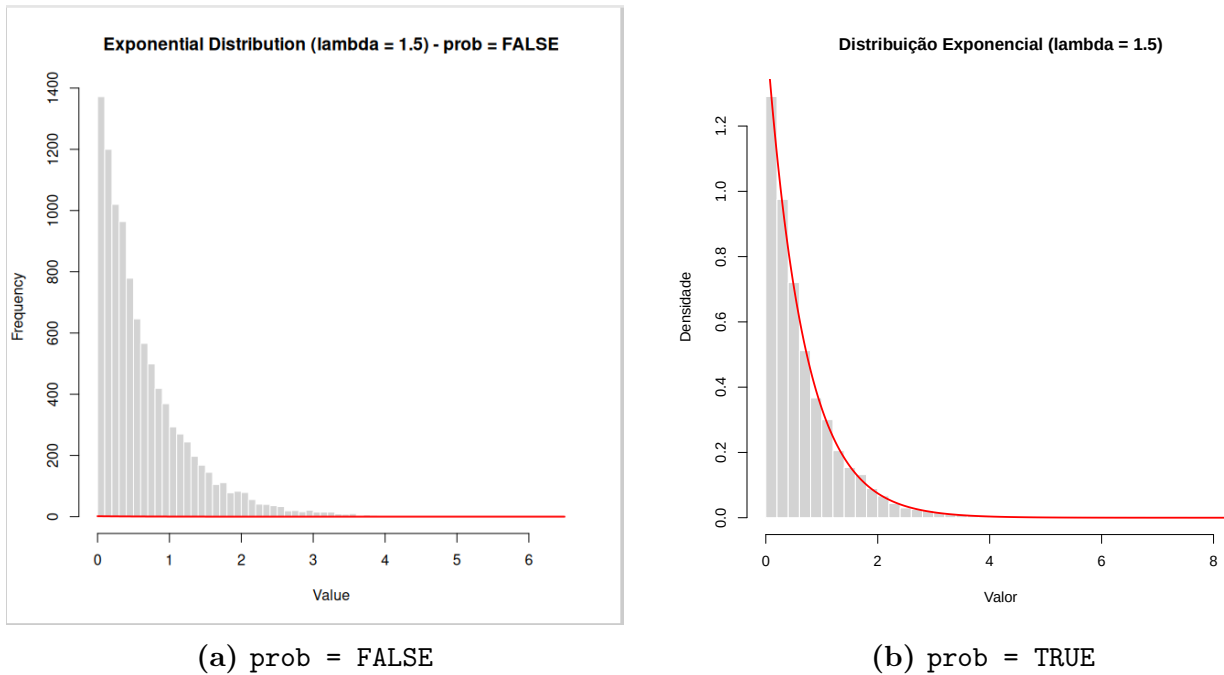


Figure 4: Histogram of 10000 exponentially distributed samples ($\lambda = 1.5$) with the theoretical pdf (red line) superposed, with and without density normalization.

Conclusion: Figure 4 shows why the `prob=TRUE` option is essential for this comparison. In Figure 4a (`prob=FALSE`), the histogram bars represent raw counts per bin, whose total sums to $n = 10000$, while the `dexp()` curve has area 1 by definition; the two are on completely different scales, so the pdf line appears flat against the axis and gives no useful comparison. In Figure 4b (`prob=TRUE`), the histogram is rescaled so that the total area of the bars is also 1, matching the normalization of the theoretical pdf, and the empirical histogram now closely follows the red curve. This confirms that the inverse transform method correctly generates exponentially distributed samples from uniform random numbers, and that density normalization is a necessary condition for a visual histogram-vs-pdf comparison to be meaningful.

3.3 Exercise 5

Objective: Develop an R script to generate the arrival times of a Poisson process, and perform a simple test to verify that the sample rate matches the population rate.

Approach: A Poisson process with rate λ has exponentially distributed inter-arrival times with parameter λ . Thus, inter-arrival times can be generated directly with `rexp()`, and the arrival times are obtained by accumulating them with `cumsum()`. To verify the generator, the sample rate is estimated from the generated data and compared with the population rate λ used to generate it.

```

lambda <- 2.0
n_arrivals <- 10000

inter_arrival_times <- rexp(n_arrivals, rate = lambda)

arrival_times <- cumsum(inter_arrival_times)

estimated_lambda <- 1/mean(inter_arrival_times)

cat("Population rate (lambda):", lambda, "\n")
cat("Estimated rate (sample):", estimated_lambda, "\n")

head(arrival_times, 10)

```

```

> n_arrivals <- 10000
>
> inter_arrival_times <- rexp(n_arrivals, rate = lambda)
>
> arrival_times <- cumsum(inter_arrival_times)
>
> estimated_lambda <- 1/mean(inter_arrival_times)
>
> cat("Taxa da população (lambda):", lambda, "\n")
Taxa da população (lambda): 2
> cat("Taxa estimada (amostral):", estimated_lambda, "\n")
Taxa estimada (amostral): 1.992062
>
> head(arrival_times, 10)
[1] 0.5065964 0.7858169 0.8949484 1.0201424 1.2737779 2.4506971 2.4521745 2.7405388 3.3030483
[10] 3.4816601
>

```

Figure 5: R console output showing the population rate, the estimated rate, and the first 10 arrival times.

Conclusion: As shown in Figure 5, with a population rate of $\lambda = 2$, the estimated rate from the sample is $\hat{\lambda} \approx 1.992$, very close to the true value. This result follows directly from the definition of λ : it represents the average number of arrivals per unit time, i.e., the rate of the process. Its reciprocal, $1/\lambda$, is the mean time between consecutive arrivals – the expected value of the exponential inter-arrival time distribution, $E[T] = 1/\lambda$. Since the sample mean of the generated inter-arrival times, $\bar{T}$, is an estimator of $E[T]$, taking its reciprocal, $1/\bar{T}$, naturally produces an estimator of λ itself. In other words, λ measures “events per unit time” while $E[T]$ measures “time per event”, so the two quantities are inverses of each other by construction, and computing $1/\bar{T}$ recovers an estimate of the rate that generated the data. The close agreement between $\hat{\lambda}$ and λ confirms that the inter-arrival times were correctly generated as exponentially distributed with rate λ .

4 C. Programming discrete event simulators

4.1 Exercise 6

Objective: Modify the M/M/1 simulator of Annex A to also estimate the average delay in system and the server utilization, and compare the results against the theoretical values (Annex B), for at least two values of λ/μ with $\lambda/\mu < 1$.

Approach: Starting from the simulator in Annex A, the following changes were made to also track the average delay in system and the server utilization, in addition to the average delay in queue already computed by the original code:

- Four new variables were initialized before the main loop: `AcumSystemTime=0` and `InServiceArrivalTime=0` to compute the delay in system (as described previously), and `BusyTime=0` together with `LastEventTime=0`, to track server utilization.
- At the very start of each loop iteration, right after `Time` is updated and *before* branching on the event type, the interval since the last event is added to `BusyTime` if the server was busy during that interval. This must be done unconditionally for both arrivals and departures, since the server can remain busy across either type of event:

```
NextEventType=which.min(EventList)
Time=EventList[NextEventType]
if (ServerStatus==1) { # NEW
  BusyTime=BusyTime+(Time-LastEventTime) # NEW
} # NEW
LastEventTime=Time # NEW
```

- In the arrival branch, when the server is free, the customer's arrival time is stored in `InServiceArrivalTime` (as before):

```
ServerStatus=1
InServiceArrivalTime=Time # NEW
EventList[2]=Time+rexp(1,ServiceRate)
```

- In the departure branch, the sojourn time of the departing customer is added to `AcumSystemTime`, and `InServiceArrivalTime` is updated if another customer enters service:

```

AcumSystemTime=AcumSystemTime+Time-InServiceArrivalTime # NEW
NumQueueCompleted=NumQueueCompleted+1
if (NumInQueue==0) {
    ServerStatus=0
    EventList[2]=Inf
} else {
    AcumDelay=AcumDelay+Time-QueueArrivalTime[1]
    InServiceArrivalTime=QueueArrivalTime[1] # NEW
    QueueArrivalTime=QueueArrivalTime[-1]
    NumInQueue=NumInQueue-1
    EventList[2]=Time+rexp(1,ServiceRate)
}

```

- After the loop, the average delay in system and the server utilization are computed alongside the average delay in queue:

```

AvgDelay=AcumDelay/NumQueueCompleted
AvgTimeInSystem=AcumSystemTime/NumQueueCompleted # NEW
Utilization=BusyTime/Time # NEW

```

The simulation was run with NumQueueCompleted = 1000 for three stable configurations ($\lambda/\mu < 1$) and, for comparison, one critically loaded configuration ($\lambda/\mu = 1$). The theoretical values were computed from Annex B: $W_Q = \frac{\lambda}{\mu(\mu - \lambda)}$, $W = \frac{1}{\mu - \lambda}$, and $\rho = \lambda/\mu$.

λ	μ	W_Q (sim.)	W_Q (theo.)	W (sim.)	W (theo.)	ρ (sim.)	ρ (theo.)
1	2	0.4572	0.5000	0.9473	1.0000	0.5421	0.5000
1	3	0.1678	0.1667	0.5142	0.5000	0.2975	0.3333
1	4	0.0727	0.0833	0.3186	0.3333	0.2732	0.2500
2	2	3.3556	∞	3.8212	∞	0.9939	1.0000

Table 1: Simulated vs. theoretical average delay in queue (W_Q), average delay in system (W), and server utilization (ρ), for different λ/μ ratios.

Conclusion: As shown in Table 1, for all stable configurations ($\rho < 1$) the simulated W_Q , W , and ρ closely match the theoretical values predicted by the M/M/1 formulas, with the small discrepancies attributable to the finite sample size (1000 completed customers) and the inherent variability of the simulation. Both the delays and the utilization increase as λ/μ approaches 1: for the M/M/1 system, $\rho = \lambda/\mu$ is precisely the theoretical fraction of time the server is busy, and this matches well against `BusyTime/Time` measured directly from the simulated trajectory. The last row ($\lambda = \mu = 2$, $\rho = 1$) illustrates the boundary of stability: the theoretical delays are infinite in the steady state, since the system is critically loaded (arrivals occur, on average, as fast as they can be served), yet the simulation returns large but finite delay values together with a simulated utilization close to 1 (0.9939). This finite delay is not a valid estimate of a steady-state quantity; it simply reflects that, over a finite run of 1000 customers, the queue built

up substantially – with the server almost always busy – without the delay yet diverging. This confirms that the condition $\rho < 1$ required by the exercise is essential for the M/M/1 system to reach a stable steady state, and shows that server utilization, unlike the mean queuing delay, remains a well-defined and finite quantity even at the stability boundary.

4.2 Exercise 7

Objective: Develop a simulator for estimating the average delay in queue of an M/M/2 system, and compare the results with the theoretical values (Annex C), for at least two values of $\lambda/(2\mu)$ with $\lambda/(2\mu) < 1$.

Approach: The M/M/1 simulator of Annex A was extended to two parallel servers. The key changes are: **ServerStatus** becomes a length-2 vector (one entry per server), and the event list has three entries instead of two – one for arrivals and one departure event per server. On arrival, if both servers are busy the customer joins the queue; otherwise the customer is assigned to the first free server found with `which(ServerStatus==0)[1]`. On a departure, the freed server either goes idle (if the queue is empty) or immediately picks up the next customer in queue, and the corresponding delay is accumulated exactly as in the M/M/1 case.

```

ArrivalRate=1
ServiceRate=2
Time=0
NumQueueCompleted=0
ServerStatus=c(0,0) # 0 = idle, 1 = busy, one entry per server
NumInQueue=0
AcumDelay=0
QueueArrivalTime=c()
EventList=c(rexp(1,ArrivalRate), Inf, Inf) # [arrival, dep@server1, dep@server2]

while (NumQueueCompleted<1000) {
  NextEventType=which.min(EventList)
  Time=EventList[NextEventType]

  if (NextEventType==1) { # ARRIVAL
    EventList[1]=Time+rexp(1,ArrivalRate)
    if (all(ServerStatus==1)) { # both servers busy -> join queue
      QueueArrivalTime=c(QueueArrivalTime,Time)
      NumInQueue=NumInQueue+1
    } else { # a server is free -> start service
      freeServer=which(ServerStatus==0)[1]
      ServerStatus[freeServer]=1
      NumQueueCompleted=NumQueueCompleted+1
      EventList[freeServer+1]=Time+rexp(1,ServiceRate)
    }
  } else { # DEPARTURE from server (NextEventType-1)
    server=NextEventType-1
    if (NumInQueue==0) {
      ServerStatus[server]=0
      EventList[NextEventType]=Inf
    } else {
      AcumDelay=AcumDelay+Time-QueueArrivalTime[1]
      QueueArrivalTime=QueueArrivalTime[-1]
      NumInQueue=NumInQueue-1
      NumQueueCompleted=NumQueueCompleted+1
      EventList[NextEventType]=Time+rexp(1,ServiceRate)
    }
  }
}

AvgDelay=AcumDelay/NumQueueCompleted

```

The simulation was run with $\text{NumQueueCompleted} = 1000$ for two configurations satisfying $\rho = \lambda/(2\mu) < 1$. The theoretical average delay in queue was computed from Annex C: $W_Q = P_Q \frac{\rho}{\lambda(1-\rho)}$, where P_Q is the probability that an arrival finds both servers busy (Erlang C formula, $m = 2$).

λ	μ	$\rho = \lambda/(2\mu)$	W_Q (sim.)	W_Q (theo.)
1	2	0.25	0.0556	0.0333
2	2	0.50	0.1380	0.1667

Table 2: Simulated vs. theoretical average delay in queue (W_Q) for the M/M/2 system, for different $\lambda/(2\mu)$ ratios.

Conclusion: As shown in Table 2, the simulated W_Q is reasonably close to the theoretical value in both configurations, with the discrepancy attributable to the finite sample size (1000 completed customers) and the inherent variability of a single simulation run – this is discussed further in Exercise 12, which shows that a single run is generally not enough to draw firm conclusions. As with the M/M/1 system, the average delay in queue increases with $\rho = \lambda/(2\mu)$, since higher load makes it more likely that an arriving customer finds both servers busy. Comparing with Exercise 6, for a similar load level the M/M/2 system exhibits substantially lower delays than the corresponding M/M/1 system (e.g., $\rho = 0.5$ gives $W_Q \approx 0.17$ here, versus $W_Q \approx 0.50$ for M/M/1 with $\lambda = 1, \mu = 2$), illustrating the benefit of server pooling: two servers each handling half the traffic intensity of an equivalent single fast server yield lower expected waiting times, since the probability of both being simultaneously busy is lower than the probability of a single server being busy.

5 D. Estimation of probabilities

5.1 Exercise 8

Objective: Estimate, through simulation, the probability of obtaining exactly two heads when five biased coins are tossed, where the probability of obtaining a head is $p = 0.7$. The simulated estimate is then compared with the exact probability.

Approach: Each coin toss was generated from a uniform random variable $U \sim \text{Unif}(0, 1)$. A toss was classified as a head when $U < p$ and as a tail otherwise. This experiment was repeated 1000 times, with five tosses in each experiment. The estimated probability is the fraction of experiments that resulted in exactly two heads. The seed was fixed to make the result reproducible.

```

p_head <- 0.7
n_coins <- 5
n_experiments <- 1000

set.seed(8)

random_values <- matrix(
  runif(n_experiments * n_coins),
  nrow = n_experiments,
  ncol = n_coins
)

coin_tosses <- random_values < p_head
number_of_heads <- rowSums(coin_tosses)

simulation_probability <- mean(number_of_heads == 2)

```

Exact probability: Let X denote the number of heads in five tosses. Since the tosses are independent and have the same probability of success, $X \sim \text{Binomial}(5, 0.7)$. Therefore,

$$P(X = 2) = \binom{5}{2} (0.7)^2 (1 - 0.7)^3 = 0.1323.$$

The simulation and exact results are shown in Table 3.

Simulation probability	Exact probability	Absolute error
0.1480	0.1323	0.0157

Table 3: Simulated and exact probability of obtaining exactly two heads.

Conclusion: The estimated probability is 0.1480, while the exact value is 0.1323. The absolute error is 0.0157, which is expected because the estimate is based on a finite number of experiments and therefore is subject to random variation. Increasing the number of repetitions would generally make the simulated probability closer to the exact value.

6 E. Revisions of statistics

6.1 Exercise 9

Objective: Analyze and compare the packet-size distributions recorded in the Wireshark captures for video, download, and normal-state network activity.

Approach: Packet sizes were read from the Wireshark CSV **Length** field. For each valid capture, missing or negative values were discarded, and packets larger than 1500 bytes were excluded. This threshold is the analysis cutoff used by the script; unusually large records can result from offloading or capture artifacts, although the cutoff may also exclude large legitimate frames depending on what the capture's **Length** field counts. The same 50-byte histogram bins

and 0–1500-byte horizontal scale were used for all captures. The main statistics were computed as follows:

```
packet_sizes <- as.numeric(capture$Length)
packet_sizes <- packet_sizes[!is.na(packet_sizes) & packet_sizes >= 0]
packet_sizes <- packet_sizes[packet_sizes <= 1500]

mean(packet_sizes)
median(packet_sizes)
```

The results for all three captures are summarized in Table 4.

Capture	Read	Excluded	Analyzed	Mean (bytes)	Median (bytes)
Video	1,820	4	1,816	641.37	477.5
Normal state	53,106	191	52,915	341.01	328
Download	1,023,788	514	1,023,274	1121.06	1304

Table 4: Packet-size statistics after excluding records larger than 1500 bytes.

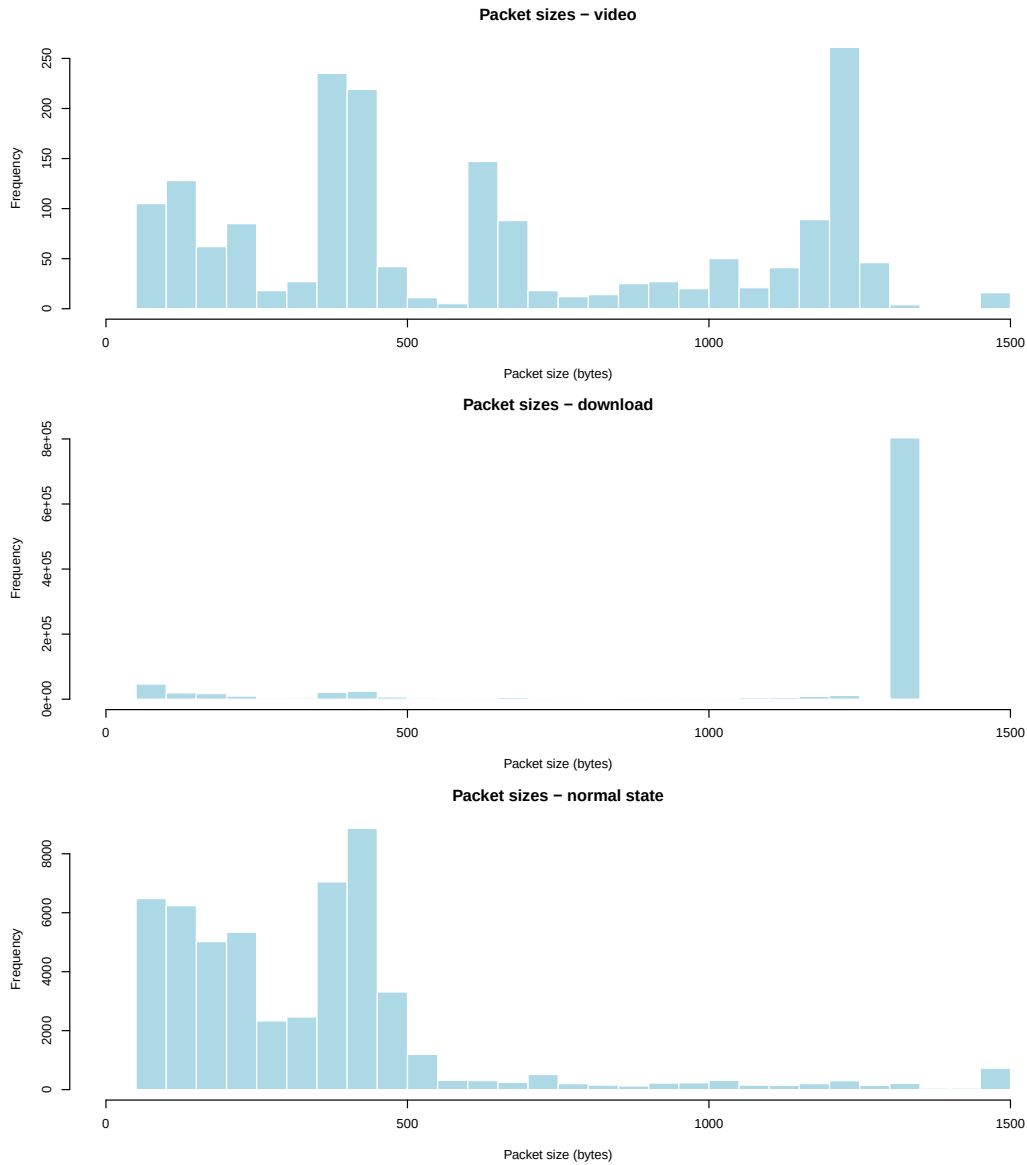


Figure 6: Packet-size histograms for the video, download, and normal-state captures, using common 50-byte bins.

Conclusion: The download capture has the largest packet sizes among the three samples, with a mean of 1121.06 bytes and a median of 1304 bytes. Video traffic is intermediate (mean 641.37 bytes; median 477.5 bytes), while normal-state traffic has the smallest values (mean 341.01 bytes; median 328 bytes). The download median exceeds its mean, unlike the other two captures, indicating that smaller observations pull its average downward even though at least half of its analyzed packets are 1304 bytes or larger. The results characterize these particular captures and do not, by themselves, establish a causal relationship between the activity and packet size.

6.2 Exercise 10

Objective: Measure 10 Round Trip Times (RTTs) to a distant country and determine two 95% confidence intervals for the average RTT, one based on the Student's t distribution and the other on the Normal distribution, discussing which is preferable.

Approach: The destination, `www.uq.edu.au` (University of Queensland, Australia), was suggested by an LLM-based chatbot that was consulted for a suitable distant host; the measurements themselves were collected by us. Ten RTTs were obtained in a single run of `ping -c 10`, all of them valid replies (Figure 7). The sample mean $\bar{x}$, standard deviation s and standard error $s/\sqrt{n}$ were computed, and the critical points were obtained with `qnorm()` and `qt()`. Both intervals have the form $\bar{x} \pm c \cdot s/\sqrt{n}$, differing only in the critical value c : $z_{0.975}$ for the Normal and $t_{0.975, n-1}$ for Student's t .

```
ataid_jo@JOSELEN2212:~/DDRS/Performance-Evaluation-and-Dimensioning-of-Networks-and-Systems$ ping -c 10 www.uq.edu.au
PING www.uq.edu.au (130.102.184.3) 56(84) bytes of data:
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=1 ttl=233 time=325 ms
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=2 ttl=233 time=317 ms
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=3 ttl=233 time=316 ms
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=4 ttl=233 time=316 ms
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=5 ttl=233 time=316 ms
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=6 ttl=233 time=317 ms
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=7 ttl=233 time=316 ms
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=8 ttl=233 time=317 ms
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=9 ttl=233 time=316 ms
64 bytes from wsa-float-prnpref.lb.dc.uq.edu.au (130.102.184.3): icmp_seq=10 ttl=233 time=321 ms

--- www.uq.edu.au ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9706ms
rtt min/avg/max/mdev = 315.744/317.745/325.308/2.977 ms
ataid_jo@JOSELEN2212:~/DDRS/Performance-Evaluation-and-Dimensioning-of-Networks-and-Systems$ |
```

Figure 7: Output of `ping -c 10 www.uq.edu.au`.

```
rtt <- c(325, 317, 316, 316, 316, 317, 316, 317, 316, 321) # RTT samples (ms)

n <- length(rtt)
xbar <- mean(rtt)
s <- sd(rtt)
se <- s / sqrt(n) # standard error of the mean

alpha <- 0.05 # 95% confidence
z_crit <- qnorm(1 - alpha/2) # Normal critical point
t_crit <- qt(1 - alpha/2, df = n - 1) # Student's t critical point

ci_normal <- c(xbar - z_crit * se, xbar + z_crit * se)
ci_t <- c(xbar - t_crit * se, xbar + t_crit * se)
```

n	$\bar{x}$ (ms)	s (ms)	$s/\sqrt{n}$ (ms)	Critical value	95% CI (ms)
10	317.70	2.98	0.94	$z = 1.960$ (Normal)	[315.85, 319.55]
10	317.70	2.98	0.94	$t_9 = 2.262$ (Student)	[315.57, 319.83]

Table 5: 95% confidence intervals for the average RTT using the Normal and Student's t distributions.

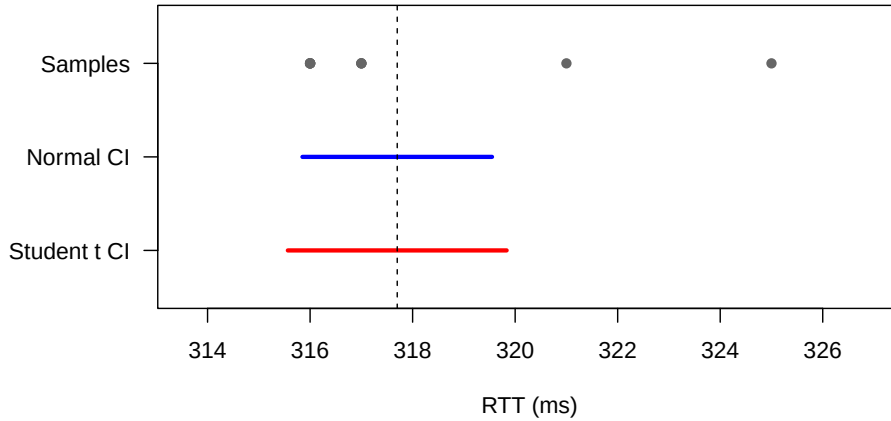


Figure 8: RTT samples and the two 95% confidence intervals (dashed line: sample mean).

Conclusion: As shown in Table 5 and Figure 8, the Student's t interval is wider than the Normal one (4.27 ms versus 3.70 ms), since $t_9 = 2.262 > z = 1.960$. The Normal interval treats s as if it were the true population standard deviation, which is only reasonable for large samples. With $n = 10$, s is itself a noisy estimate, and the t distribution compensates for this extra uncertainty with heavier tails; the difference vanishes as n grows. The Normal interval is therefore too optimistic and would cover the true mean less than 95% of the time, so we prefer the Student's t interval. In this run the RTTs are very stable: eight of the ten lie between 316 and 317 ms, with two higher values (321 and 325 ms) pulling the mean slightly above the cluster. As a result s is small and both intervals are narrow, differing by less than 0.3 ms at each end. The data are not Normal (they are skewed to the right), and since the t interval assumes approximately Normal data, it should be interpreted with some caution; this is investigated in Exercise 11, which assesses the actual coverage of these intervals.

6.3 Exercise 11

7 F. Output data analysis

7.1 Exercise 12

Objective: Assess the variability of the M/M/1 average queue-delay estimate by running the simulator ten times, with 1000 completed customers per run and traffic intensity $\rho = \lambda/\mu = 0.8$, and determine whether a single simulation is sufficient to draw conclusions.

Approach: The M/M/1 event-driven simulator from Annex A was executed independently ten times. We chose $\lambda = 0.8$ and $\mu = 1$, which gives $\rho = 0.8$. For each run, the average queue

delay was calculated as the total time customers spent waiting in the queue divided by the number of completed customers; customers who entered service immediately contribute zero waiting time to the average. A fixed random seed (12) makes the experiment reproducible. The simulation loop was repeated as follows:

```
set.seed(12)
queue_delay_estimates <- replicate(
  10,
  simulate_mm1_queue_delay(
    arrival_rate = 0.8,
    service_rate = 1,
    completed_customers = 1000
  )
)
```

The theoretical steady-state average queue delay is

$$W_Q = \frac{\lambda}{\mu(\mu - \lambda)} = \frac{0.8}{1(1 - 0.8)} = 4.$$

The ten simulation estimates are shown in Table 6.

Run	W_Q estimate	Run	W_Q estimate
1	2.7210	6	2.3488
2	5.7114	7	4.3678
3	2.7891	8	3.5047
4	3.7936	9	2.4763
5	2.7548	10	3.9490

Table 6: Average queue-delay estimates from ten independent M/M/1 simulation runs (1000 completed customers per run).

Across the ten replications, the sample mean was 3.4417 and the sample standard deviation was 1.0508. The estimates ranged from 2.3488 to 5.7114. A 95% Student's t confidence interval for the mean across replications was [2.6899, 4.1934], which contains the theoretical value $W_Q = 4$.

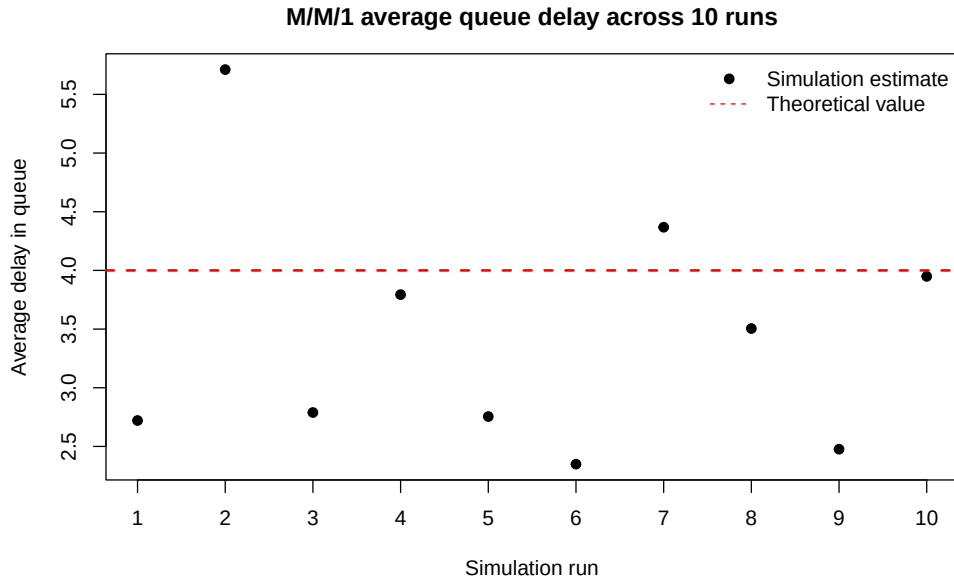


Figure 9: Average queue-delay estimate in each of the ten M/M/1 runs. The dashed line marks the theoretical value $W_Q = 4$.

Conclusion: The estimates differ noticeably from run to run even though each run processes 1000 completed customers. In this experiment they range from approximately 2.35 to 5.71, so one run is not sufficient to characterize the estimator’s variability or to make a robust conclusion from a single observed value. The mean of the ten estimates is below the theoretical value, but the 95% confidence interval across replications includes 4, which is consistent with the theoretical result at this confidence level. More independent replications, or longer runs, would provide a more precise assessment; this single set of ten runs illustrates why replication is important.

7.2 Exercise 13

Objective: Evaluate how the initial number of customers in an M/M/1 system affects estimates of the average delay in queue. Twenty-five independent runs were performed for each combination of run length (20 or 2000 completed customers) and initial population (0 or 10 customers), with $\lambda = 3$ and $\mu = 4$.

Approach: Each run used the event-driven M/M/1 simulator. With ten customers initially in the system, one customer starts in service and nine are waiting in the queue; their initial arrival times are set to time zero. Thus, their waiting times are measured from the start of the simulation. The run stops after the specified number of completed customers, and the average queue delay is calculated over those completed customers. The supplied `initcond.R` helper was not present in the repository, so the script initializes this state directly. The four experimental conditions were generated as follows:

```
run_results <- expand.grid(
  run = 1:25,
  run_length = c(20, 2000),
  initial_clients = c(0, 10)
)
```

For $\rho = \lambda/\mu = 0.75$, the theoretical steady-state mean delay in queue is

$$W_Q = \frac{\lambda}{\mu(\mu - \lambda)} = \frac{3}{4(4 - 3)} = 0.75.$$

Table 7 reports the mean of the 25 simulation estimates for each condition and the corresponding 95% Student's t confidence interval across replications.

Completed customers	Initial customers	Mean W_Q	95% confidence interval
20	0	0.4637	[0.2697, 0.6578]
20	10	1.4241	[1.2124, 1.6357]
2000	0	0.7159	[0.6633, 0.7684]
2000	10	0.7373	[0.6765, 0.7980]

Table 7: M/M/1 average queue-delay estimates over 25 runs per condition ($\lambda = 3$, $\mu = 4$).

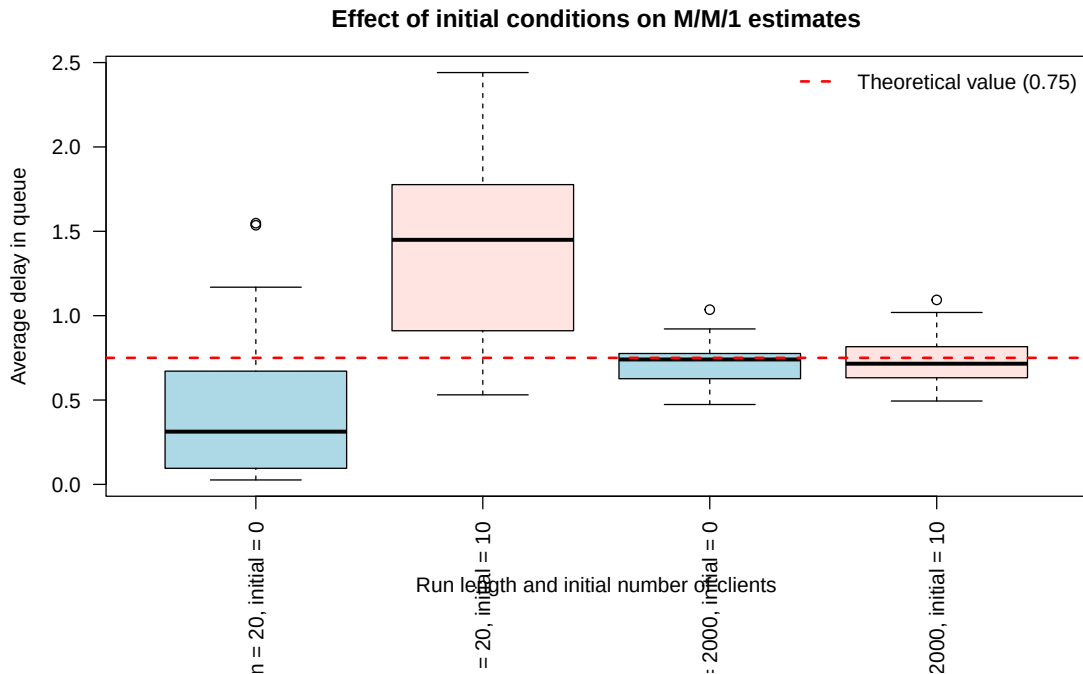


Figure 10: Distribution of the 25 average queue-delay estimates for each run length and initial condition. The dashed line marks the theoretical value $W_Q = 0.75$.

Conclusion: The initial condition has a pronounced effect when only 20 customers are used. Starting empty produces a mean estimate below steady state (0.4637), while starting

with ten customers produces a much higher estimate (1.4241), because the initial backlog contributes large waiting times. Neither 95% interval includes the theoretical value 0.75. With 2000 completed customers, the estimates are much closer to theory (0.7159 and 0.7373), and both intervals include 0.75. This indicates that a longer run reduces the relative influence of the initial transient, while short runs can give strongly biased estimates depending on the initial state.

7.3 Exercise 14

Objective: Study how the number of independent replications required to estimate the M/M/1 average queue delay depends on the target relative error, the number of measured delays per replication, and the traffic intensity $\rho = \lambda/\mu$.

Approach: The provided `repdel.R` file was not present in the repository, so a self-contained equivalent of `frepdel()` was implemented. For every replication, the first 10000 queue delays were discarded to reduce the empty-system initialization transient; the estimate was then calculated from either 100 or 1000 subsequent completed customers. Starting with ten replications, independent replications were added until the relative half-width of the 95% Student's t confidence interval satisfied the target:

$$\frac{t_{0.975, R-1} s_R}{\sqrt{R} |\bar{W}_{Q,R}|} \leq e,$$

where R is the number of replications, s_R is the sample standard deviation of the replication estimates, and e is the requested relative error. We used $\mu = 1$, $\rho \in \{0.50, 0.80, 0.95\}$, measured sample sizes $n \in \{100, 1000\}$, and target errors $e \in \{5\%, 10\%, 20\%\}$. The theoretical mean and variance of one steady-state M/M/1 queue delay are

$$E[W_Q] = \frac{\rho}{\mu - \lambda}, \quad \text{Var}(W_Q) = \frac{\rho(2 - \rho)}{(\mu - \lambda)^2}.$$

For $\mu = 1$, the theoretical variances are 3, 24, and 399 for $\rho = 0.50, 0.80$, and 0.95 , respectively. Figure 11 shows both this increase and the observed replication requirements.

ρ	Delays per replication	$e = 5\%$	$e = 10\%$	$e = 20\%$
0.50	100	406	101	34
0.80	100	1013	223	59
0.95	100	1643	417	93
0.50	1000	44	18	10
0.80	1000	180	14	10
0.95	1000	1059	208	45

Table 8: Number of replications required to reach each relative-error target using a 95% confidence interval. At least ten replications were used in every case.

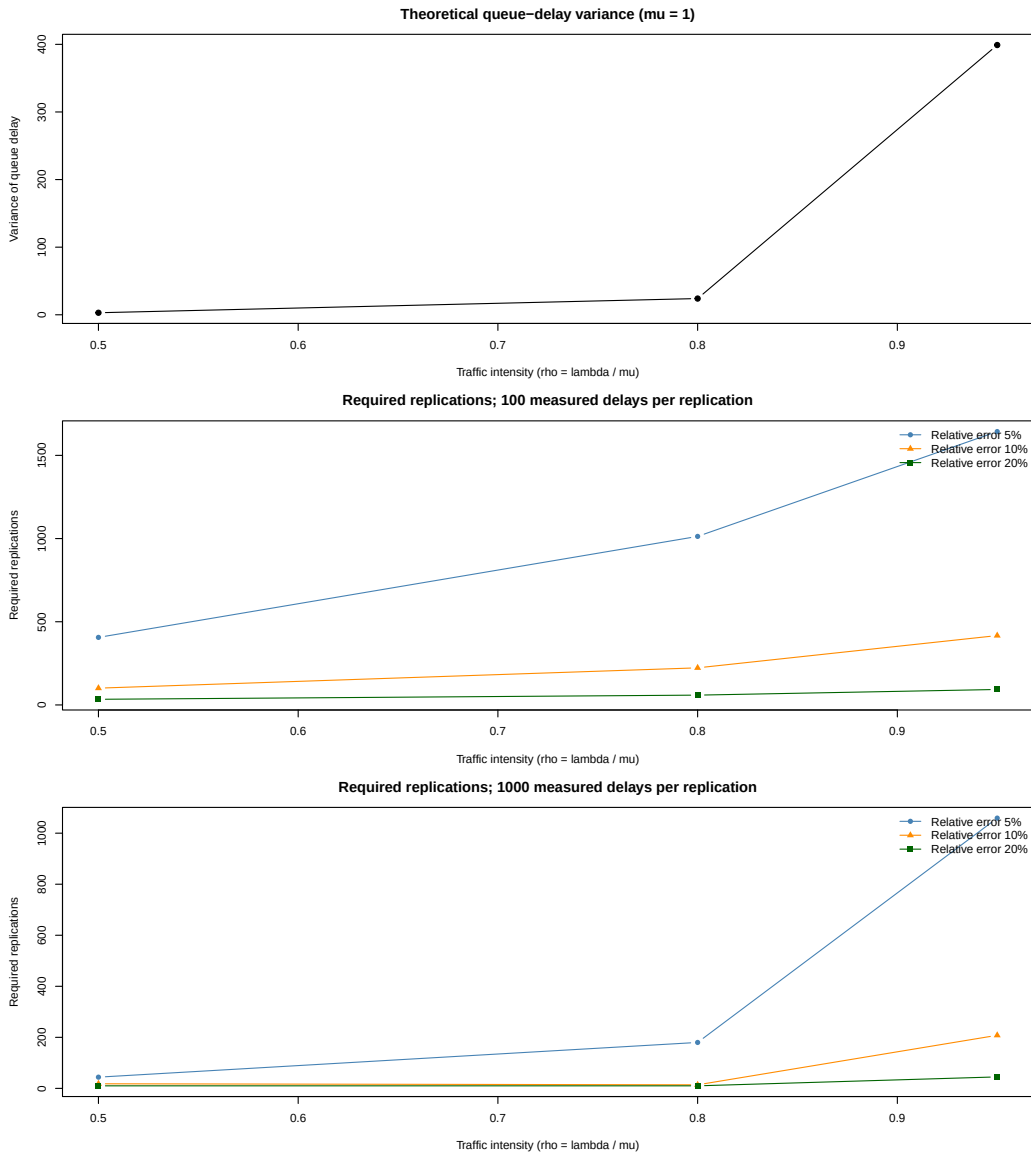


Figure 11: Theoretical queue-delay variance and observed replication requirements as functions of ρ , for two measured sample sizes.

Conclusion: The required number of replications increases as the relative-error target becomes stricter. Increasing the number of measured delays per replication generally reduces the required number of replications, since each replication estimate is more precise. The required count also rises sharply as ρ approaches one; for example, with 100 measured delays and a 5% target, it increases from 406 at $\rho = 0.50$ to 1643 at $\rho = 0.95$. This is consistent with the strong increase in queue-delay variance near saturation. The results therefore show that both the desired precision and system load must be considered when choosing the number of replications. Counts are specific to the stated seed, warm-up deletion, and tested grid; they are simulation estimates rather than universal constants.

8 Conclusion

Text for the conclusion here.